Lecture 8: Graph Algorithms

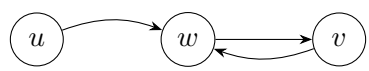
Connectivity in Digraphs · SCCs · Shortest Paths · Dijkstra's Algorithm

1 Connectivity in Directed Graphs

Throughout, $G = (V, E)$ denotes a directed graph (digraph) with vertex set V and edge set E.

Definition 1.1 (Reachability)**.** In a digraph, we say vertex v is **reachable** from vertex u if there exists a directed path from u to v.

Note: Reachability is *not* symmetric. The figure below illustrates this: v is reachable from u (following the path $u \to w \to v$), but u is not reachable from v (there is no directed path back).



*Reachability is **not** symmetric in a digraph.*

Definition 1.2 (Strong Connectivity)**.** In a digraph, vertices u and v are **strongly connected** if u is reachable from v *and* v is reachable from u.

Strong connectivity is an equivalence relation. It is:

- **Reflexive**: every vertex is reachable from itself (via the empty path).
- **Symmetric**: by definition, $u \leftrightarrow v$ implies $v \leftrightarrow u$.
- **Transitive**: if $u \leftrightarrow v$ and $v \leftrightarrow w$, then $u \leftrightarrow w$.

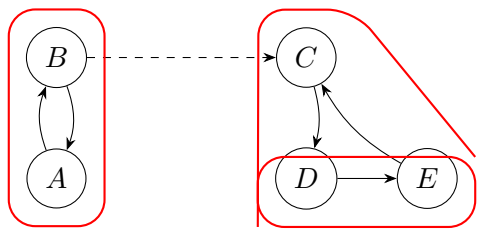
Because it is an equivalence relation, it partitions V into disjoint *equivalence classes*.

2 Strongly Connected Components (SCCs)

Definition 2.1 (Strongly Connected Component)**.** A **strongly connected component (SCC)** of a digraph is a *maximal* subset $C \subseteq V$ such that every pair of vertices $u, v \in C$ are strongly connected.

"Maximal" means we cannot add any more vertices to C while preserving the property.

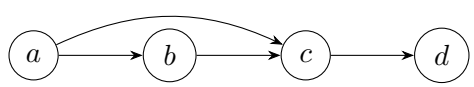
Example 2.1. Consider the following digraph:



The SCCs are $\{A, B\}$ and $\{C, D, E\}$, with a cross-edge from the first to the second.

Example 2.2 (SCCs of a DAG)**. Q: What are the SCCs of a DAG (directed acyclic graph)?**

In a DAG there are no directed cycles, so no two distinct vertices can be mutually reachable. Therefore, **every vertex is its own SCC** (a singleton).



Each node is its own SCC (no back edges exist in a DAG).

2.1 Decomposition of Digraphs

Definition 2.2 (Metagraph / Condensation)**.** Given a digraph G, form the **metagraph** G' (also called the *condensation*) in which:

- Each SCC of G is represented by a single *supernode*.
- There is a directed edge (X, Y) in G' if there exist some vertex $u \in X$ and some vertex $v \in Y$ such that $(u, v) \in E$.

Fact 2.1. *G' is always a DAG.*

Intuition: A cycle in G' would correspond to a cycle among SCCs, which would force those SCCs to merge into one — contradicting maximality.

Key structural result: Every directed graph is a DAG of its strongly connected components.

2.2 Linear-Time SCC Decomposition

Algorithm 1 Linear-Time SCC Decomposition (Kosaraju's Algorithm)

1: Run DFS on the *reverse graph* G^R, where every edge of G has its direction reversed.
2: Order the vertices of G by **decreasing post-order finish time** from step 1.
3: Run DFS on the original graph G using this vertex order. Process vertices in the order determined in step 2.
4: Each time the DFS stack empties (i.e., when a DFS tree is complete), the vertices explored form one SCC. Remove this component and record it.

Claim 2.1. *The components produced by Kosaraju's algorithm are exactly the SCCs of G, and they appear in reverse topological order of the metagraph G'.*

Proof: Left as an exercise (homework).

Running time: The algorithm runs two iterations of DFS.

$$O(n + m)$$

where $n = |V|$ and $m = |E|$.

3 Shortest Paths

Let $G = (V, E)$ be a directed graph with real-valued edge weights $\{w_e\}_{e \in E}$ assigned to the edges.

Definition 3.1 (Path Length)**.** The **length** (or *weight*) of a path $P = (v_1, v_2, \ldots, v_k)$ is:

$$w(P) = \sum_{i=1}^{k-1} w_{(v_i,\, v_{i+1})}$$

Example 3.1. For the path $u \xrightarrow{3} w \xrightarrow{-1} v \xrightarrow{2} \cdots$, the weight is $w(P) = 3 + (-1) + 2 = 4$.

Definition 3.2 (Shortest Path Length)**.** The **shortest path length** from u to v, denoted $d(u, v)$, is the minimum weight over all directed paths from u to v:

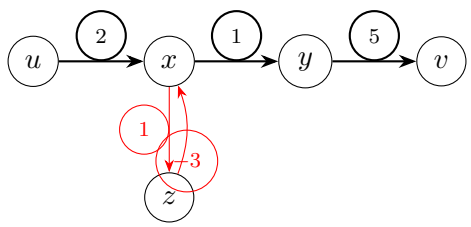
$$d(u, v) = \min_{P:\, u \rightsquigarrow v} w(P)$$

A **shortest path** is any path that achieves this minimum.

3.1 When Does a Shortest Path Exist?

Q: Does a shortest path always exist? No — there are two obstruction cases:

1. **Disconnected vertices.** If there is no directed path from u to v, we set $d(u, v) = +\infty$.
2. **Negative-weight cycles.** If there is a negative-weight cycle reachable from u on a path to v, then the shortest path length is $-\infty$, since we can traverse the cycle arbitrarily many times to make the total weight as small as desired.



Above: the cycle $x \to z \to x$ has total weight $1 + (-3) = -2 < 0$, so traversing it repeatedly gives paths of arbitrarily negative weight. Hence $d(u, v) = -\infty$.

3.2 The Shortest Paths Problem (Algorithmic Formulation)

Definition 3.3 (Shortest Paths Problem)**. Input:** A directed graph $G = (V, E)$ with non-negative edge weights and a distinguished source vertex $s \in V$.

Output: For every vertex $v \in V$, the shortest path length $d(s, v)$ and an actual shortest path from s to v.

Output representation: We could store for each v the entire list of edges on the shortest path, but this takes $O(n^2)$ space in the worst case. A more efficient representation uses *predecessor pointers*:

$$\text{pred}(v) = \text{the vertex immediately before } v \text{ on the shortest path from } s \text{ to } v$$

This takes only $O(n)$ space and implicitly encodes all shortest paths (by following pred pointers back to s).

3.3 Structural Property: Nested Subpath Optimality

Theorem 3.1 (Optimal Substructure of Shortest Paths)**.** *Every subpath of a shortest path is itself a shortest path. That is, if $P = (v_1, \ldots, v_k)$ is a shortest path from v_1 to v_k, then for any $1 \le i \le j \le k$, the sub-path $(v_i, \ldots, v_j)$ is a shortest path from v_i to v_j.*

Proof sketch: If some subpath were not optimal, we could replace it with a shorter one, contradicting the assumption that the full path was shortest.

4 Dijkstra's Algorithm

4.1 Key Observation

Basic observation. For any edge $(v, w) \in E$, shortest path distances must satisfy:

$$d(s, v) + w_{vw} \;\geq\; d(s, w)$$

for all edges $(v, w) \in E$. If this inequality is *violated*, the edge (v, w) is called **incorrect**, and we must update $d(s, w)$.

We maintain:

- **Tentative distances** $d(v)$ for each $v \in V$.
- **Tentative predecessors** $\text{pred}(v)$ for each $v \in V$.

Edge correction subroutine. For an edge (v, w), we say it is *incorrect* if:

$$d(v) + w_{vw} < d(w)$$

To **correct** (v, w):

1. Set $d(w) \leftarrow d(v) + w_{vw}$
2. Set $\text{pred}(w) \leftarrow v$

Algorithm 2 Scan(v) Subroutine

```
for each edge (v, w) ∈ E do
    if (v, w) is incorrect then
        Correct (v, w)
    end if
end for
```

4.2 Full Algorithm

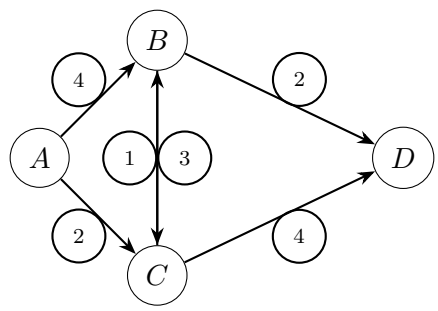
Intuition: We greedily process vertices in order of their current tentative distance. Because edge weights are non-negative, once a vertex is scanned, its distance cannot improve (no shorter path can arrive later), so we never need to revisit it.

Algorithm 3 Dijkstra's Algorithm

```
Set d(s) ← 0
Set d(v) ← +∞ for all v ∈ V \ {s}
Set pred(v) ← ∅ for all v ∈ V
Set S ← V     (S is the set of "unscanned" vertices)
while ∃ v ∈ S with d(v) < +∞ do
    Choose such a v with minimum d(v)
    Scan(v)
    S ← S \ {v}
end while
```

4.3 Worked Example

Consider the graph with source $s = A$:



Vertex	$d(\cdot)$ initial	$d(\cdot)$ final	pred$(\cdot)$ final
A	0	0	$\emptyset$
B	$+\infty$	3	C
C	$+\infty$	2	A
D	$+\infty$	5	B

Step-by-step trace:

1. **Scan(A):** A has $d(A) = 0$, minimum in $S = \{A, B, C, D\}$.

   - Edge (A, B): $0 + 4 < +\infty \Rightarrow$ correct (A, B): $d(B) = 4$, pred$(B) = A$.
   - Edge (A, C): $0 + 2 < +\infty \Rightarrow$ correct (A, C): $d(C) = 2$, pred$(C) = A$.

   Remove A from S. Now $S = \{B, C, D\}$.

2. **Scan(C):** C has $d(C) = 2$, minimum in S.

   - Edge (C, D): $2 + 4 < +\infty \Rightarrow$ correct (C, D): $d(D) = 6$, pred$(D) = C$.
   - Edge (C, B): $d(C) + w_{CB} = 2 + 1 = 3 < 4 = d(B) \Rightarrow$ correct (C, B): $d(B) = 3$, pred$(B) = C$.

Remove C from S. Now $S = \{B, D\}$.

3. **Scan(B):** B has $d(B) = 3$, minimum in S.

   - Edge (B, C): $3 + 3 = 6 \not< 2 = d(C) \Rightarrow$ already correct.
   - Edge (B, D): $3 + 2 = 5 < 6 = d(D) \Rightarrow$ correct (B, D): $d(D) = 5$, $\text{pred}(D) = B$.

   Remove B from S. Now $S = \{D\}$.

4. **Scan(D):** D has $d(D) = 5$. No outgoing edges. Remove D. $S = \emptyset$. Done.

Final shortest paths from A:

$$A \to C: \text{ length } 2, \quad A \to C \to B: \text{ length } 3, \quad A \to C \to B \to D: \text{ length } 5$$

4.4 Running Time

Naive implementation: The dominant cost is the selection of the minimum-$d(v)$ vertex (line 6).

- We perform this minimum selection $O(n)$ times.
- A naive linear scan takes $O(n)$ per selection.
- Total: $O(n^2)$ for minimum selections, plus $O(m)$ for edge relaxations.

$$T_{\text{naive}} = O(n^2 + m)$$

Better implementation with a priority queue: Using a binary min-heap or Fibonacci heap:

$$T_{\text{priority queue}} = O((n \log n + m))$$

(Using a Fibonacci heap achieves $O(n \log n + m)$; a binary heap gives $O((n + m) \log n)$.) This improvement matters when the graph is sparse ($m \ll n^2$). See textbook for details if interested.

5 Correctness of Dijkstra's Algorithm

Theorem 5.1 (Correctness of Dijkstra's Algorithm)**.** *When Dijkstra's algorithm terminates (assuming all edge weights are non-negative), $d(v)$ equals the true shortest path distance from s to v for all $v \in V$, and* $\text{pred}(\cdot)$ *encodes a shortest path tree.*

Proof by induction.

Inductive hypothesis: After each iteration of the while-loop that removes vertex v from S, the following hold simultaneously:

(a) For all $v \notin S$ (i.e., already scanned vertices), $d(v)$ is the true shortest path distance from s to v, and $\text{pred}(\cdot)$ defines a shortest path.

(b) For all remaining $v \in S$, $d(v)$ is the shortest path distance from s to v using *only* intermediate vertices in $V \setminus S$ (i.e., vertices already scanned).

Base case: Initially, $S = V$, $d(s) = 0$ (correct), and $d(v) = +\infty$ for all others (vacuously correct — no paths with no intermediate vertices).